

[CS230] Deep Neural Networks: Dimensions & Initialization

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-3. Foundations & Shallow Networks - *Completed*

Chapter 4. Deep Neural Networks

- 4.1 Deep L-Layer Neural Network Architecture - *Completed*
- **4.2 Dimensions & Initialization (Current Unit)**
 - * The Law of Matrix Dimensions
 - * Symmetry Breaking (Why not Zero?)
 - * He Initialization (The Standard for ReLU)
 - * Implementation & Verification
- 4.3 Building a Deep Neural Network Application - *Upcoming*

지난 시간 복습 및 연결

우리는 이제 거대한 심층 신경망을 설계할 수 있는 건축가가 되었습니다. 하지만 설계도만 그렸을 뿐, 아직 착공도 하지 않았습니다. 건물을 올리기 전에 가장 먼저 해야 할 일은 무엇일까요? 설계도 검증(차원 확인)과 기초 공사(초기화)입니다. 이 두 가지를 소홀히 하면 코드를 실행하자마자 에러가 터지거나(Dimension Mismatch), 에러 메시지 하나 없이 학습이 전혀 안 되는(Bad Initialization) 침묵의 버그를 만나게 됩니다.

1 Unit Overview

□ 핵심 목표

이 단원은 디버깅 시간을 획기적으로 줄여주는 '차원 분석'과 학습 성공의 열쇠인 '파라미터 초기화'를 다룹니다.

- **분석:** L 층 신경망의 파라미터(W, b)와 데이터(Z, A)의 형상(Shape)을 정확히 도출합니다.
- **이유:** 가중치를 0으로 초기화했을 때 발생하는 '대칭성 문제(Symmetry Problem)'를 증명합니다.
- **해결:** ReLU를 위한 표준 초기화 방법인 **He Initialization**의 원리와 코드를 익힙니다.

2 Essential Terminology

용어	설명	핵심 포인트
Shape	행렬의 차원 (행, 열)	디버깅의 90%는 Shape 맞추기입니다.
Symmetry Breaking	대칭성 파괴	뉴런들이 서로 다르게 학습되도록 초기값을 다르게 주는 것.
Zero Init	0으로 초기화	모든 뉴런이 똑같이 동작하게 만드는 최악의 방법.
He Init	He 초기화	ReLU 사용 시 분산을 유지해주는 최고의 방법.
Xavier Init	Xavier 초기화	Sigmoid/Tanh 사용 시 적합한 초기화 방법.

3 Core Concepts: 디버깅을 위한 헌법

3.1 1. Matrix Dimensions Rules (차원의 법칙)

코딩하다 헛갈릴 때마다 이 표를 보십시오. $n^{[l]}$ 은 현재 층의 뉴런 수, m 은 데이터 개수입니다.

Shape Cheat Sheet

- 파라미터 (학습 대상):
 - $W^{[l]}: (n^{[l]}, n^{[l-1]}) \rightarrow$ (현재 층, 이전 층)
 - $b^{[l]}: (n^{[l]}, 1) \rightarrow$ 열 벡터 (Column Vector)
- 데이터 흐름 (Activations):
 - $Z^{[l]}, A^{[l]}: (n^{[l]}, m)$
 - $dZ^{[l]}, dA^{[l]}: (n^{[l]}, m) \rightarrow$ 원래 데이터와 Shape 동일

3.2 2. Why Not Zero Initialization? (0 초기화의 저주)

”교수님, 로지스틱 회귀에선 0으로 해도 잘 됐잖아요?” 네, 하지만 신경망(은닉층이 있는 경우)에서는 절대 안 됩니다.

□ 복제 인간 군대 비유 (직관적 비유)

- 상황: 모든 가중치 W 를 0으로 초기화했습니다.
- Forward: 모든 은닉 뉴런이 입력값에 상관없이 똑같은 값(0)을 계산합니다.
- Backward: 모든 뉴런이 똑같은 오차(Gradient)를 보고받습니다.
- Update: 모든 뉴런이 똑같은 값으로 수정됩니다.
- 결과: 뉴런이 100만 개여도, 결국 뉴런 1개짜리 선형 모델과 똑같이 행동합니다. 이를 대칭성(Symmetry) 문제라고 하며, 학습이 실패합니다.

3.3 3. Best Practice: He Initialization

그렇다면 랜덤하게(Random) 초기화하면 될까요? 너무 크면(x10) 기울기 소실이 오고, 너무 작으면(x0.0001) 신호가 죽어버립니다.

Kaiming He 박사가 제안한 **He Initialization**은 ReLU를 사용할 때 분산을 일정하게 유지해주는 마법의 공식입니다.

$$W^{[l]} \sim \text{Random} \times \sqrt{\frac{2}{n^{[l-1]}}}$$

- 이전 층의 뉴런 개수($n^{[l-1]}$)가 많을수록, 가중치를 더 작게 만들어 줍니다.
- $\sqrt{2}$ 는 ReLU가 음수 영역을 0으로 만들어 분산을 절반으로 깎아먹는 것을 보상해줍니다.

4 Implementation: Initialization Strategies

나쁜 예(Zero, Large Random)와 좋은 예(He)를 코드로 비교해봅시다.

```
1 import numpy as np
2
3 class Initializer:
4     def __init__(self, layer_dims):
5         self.layer_dims = layer_dims # 예: [1000, 100, 10]
6         self.L = len(layer_dims) - 1
7
8     def init_zeros(self):
9         """
10         BAD: 모든 가중치를 0으로 초기화 -> 학습 불가
11         """
12         params = {}
13         for l in range(1, self.L + 1):
14             params['W' + str(l)] = np.zeros((self.layer_dims[l], self.layer_dims[l-1]))
15             params['b' + str(l)] = np.zeros((self.layer_dims[l], 1))
16         return params
17
18     def init_he(self):
19         """
20         BEST: He Initialization (Standard for ReLU)
21         """
22         params = {}
23         for l in range(1, self.L + 1):
24             # 1. 차원 정의
25             n_curr = self.layer_dims[l]
26             n_prev = self.layer_dims[l-1]
27
28             # 2. He Initialization 공식 적용
29             # np.random.randn: 평균 0, 분산 1인 정규분포
30             # scaling: 분산을 2/n_prev로 맞춰줌
31             scaling = np.sqrt(2 / n_prev)
32
33             params['W' + str(l)] = np.random.randn(n_curr, n_prev) * scaling
34             params['b' + str(l)] = np.zeros((n_curr, 1)) # 편향은 이어도 0 됨!
35
36         return params
37
38 # --- 검증 ---
39 if __name__ == "__main__":
40     dims = [1000, 100, 10]
41     init = Initializer(dims)
42
```

```

43 # He Init 결과 확인
44 params = init.init_he()
45 W1 = params['W1']
46
47 print("Shape Check:", W1.shape) # (100, 1000)
48 print("Variance Check:", np.var(W1))
49 print("Expected Variance:", 2/1000) # 0.002 근처여야 함

```

Listing 1: Parameter Initialization Methods

5 FAQ & Pitfalls

편향(Bias) b 는 0으로 해도 되나요? (오해 방지 가이드)

네, 됩니다! 대칭성 문제는 가중치 W 에서 발생합니다. W 가 이미 랜덤하게 섞여 있다면 (Symmetry Broken), 편향 b 가 모두 0이어도 뉴런들은 서로 다른 값을 출력하게 됩니다. 따라서 b 는 편의상 'np.zeros'로 초기화하는 것이 일반적입니다.

Q. Xavier 초기화는 뭔가요?

A. Sigmoid나 Tanh 함수를 쓸 때 사용하는 초기화 방법입니다. 계수가 $\sqrt{1/n}$ 입니다. 하지만 요즘 딥러닝은 대부분 ReLU를 쓰기 때문에 He 초기화($\sqrt{2/n}$)가 더 많이 쓰입니다.

다음 단계 (Next Step)

이제 우리는 설계(Architecture), 기초 공사(Initialization), 자재 검수(Dimension Check)까지 완벽하게 마쳤습니다.

이제 남은 것은 건물을 짓는 것뿐입니다. 다음 시간에는 [Project] Building a Deep Neural Network Application을 통해, 우리가 만든 코드로 '고양이 vs 개' 이미지를 분류하는 인공지능을 완성하겠습니다. 여러분의 첫 번째 Deep Learning 프로젝트가 시작됩니다!

□ 단원 요약 (Cheat Sheet)

1. **Dimensions:** W 는 $(n^{[l]}, n^{[l-1]})$ 이다. 차원 확인이 디버깅의 시작이다.
2. **Zero Init:** W 를 0으로 하면 학습이 안 된다. (대칭성 문제)
3. **He Init:** ReLU를 쓸 때는 'randn * sqrt(2/n)' 공식을 사용하라.
4. **Bias:** 편향 b 는 0으로 초기화해도 안전하다.